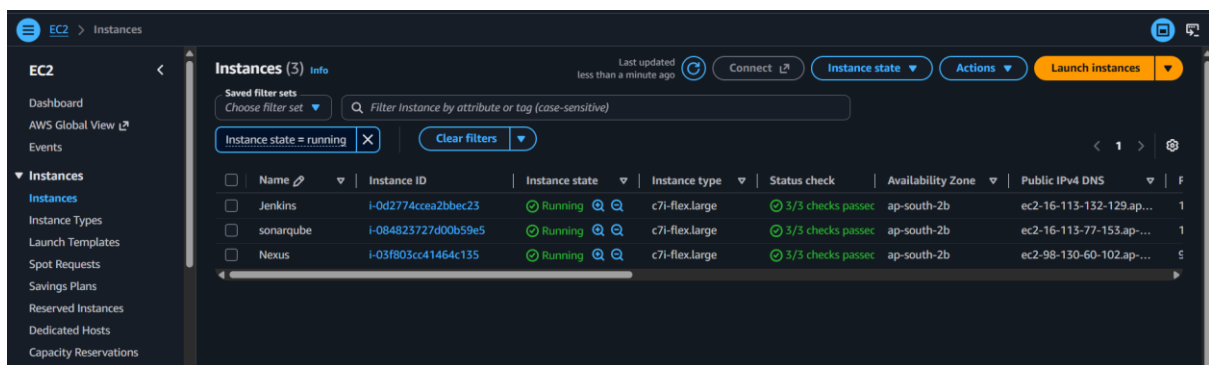


Docker Web Stack – CI/CD Project Overview

Docker Web Stack is a Java-based web application that provides user-oriented functionality such as user registration, login, session management, and logout. The project is used to implement a complete DevOps CI/CD pipeline, starting from source code management and application build through code-quality analysis, artifact management, containerization, image publishing, and deployment using Docker Swarm.

The application is built using Java and Maven, analyzed using SonarQube, packaged as a WAR artifact, stored in Nexus Repository, containerized using Docker, published to Docker Hub, and finally deployed as a multi-service application using Docker Swarm and Docker Stack.

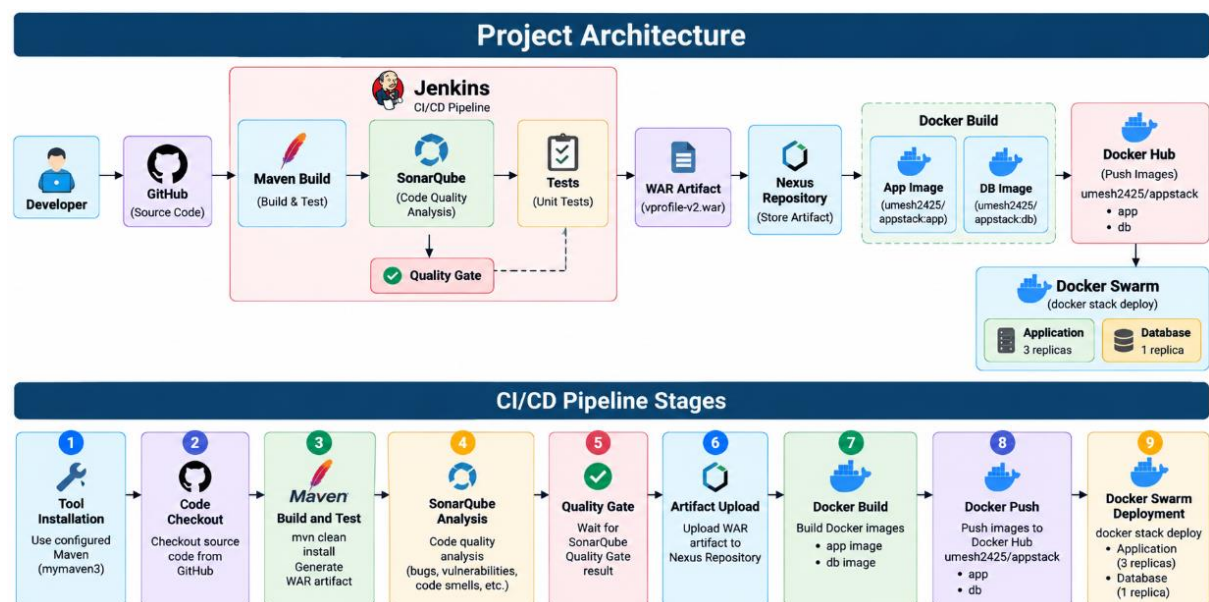
Servers



The screenshot shows the AWS Management Console 'Instances' page. It lists three EC2 instances: Jenkins, sonarqube, and Nexus. All instances are in a 'Running' state, using the 'c7i-flex.large' instance type, and are located in the 'ap-south-2b' availability zone. Each instance has a 'Status check' of '3/3 checks passed'.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Count
Jenkins	i-0d2774ccea2bbec23	Running	c7i-flex.large	3/3 checks passed	ap-south-2b	ec2-16-113-132-129.ap...	1
sonarqube	i-084823727d00b59e5	Running	c7i-flex.large	3/3 checks passed	ap-south-2b	ec2-16-113-77-153.ap...	1
Nexus	i-03f803cc41464c135	Running	c7i-flex.large	3/3 checks passed	ap-south-2b	ec2-98-130-60-102.ap...	5

Project Architecture



Jenkins Server Installation

sudo yum install java-21-amazon-corretto -y

```
[root@jenkins ~]# sudo yum install java-21-amazon-corretto -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
```

java -version

openjdk version "21.0.12.1" 2026-08-18 LTS

OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)

OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)

```
[root@jenkins ~]# java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
```

**sudo wget -O /etc/yum.repos.d/jenkins.repo **

<https://pkg.jenkins.io/rpm-stable/jenkins.repo>

```
[root@jenkins ~]# sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo
--2026-10-03 08:29:05-- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 199.232.106.133, 2a04:4e42:5a::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|199.232.106.133|:443... connected.
```

sudo yum install Jenkins

```
[root@jenkins ~]# sudo yum install jenkins
jenkins-stable                               18 kB/s | 833 B      00:00
jenkins-stable                               438 kB/s | 1.6 kB    00:00
Importing GPG key 0x14ABFC68:
```

jenkins --version

```
[root@jenkins ~]# jenkins --version
2.580.1
```

2.580.1

sudo systemctl start Jenkins

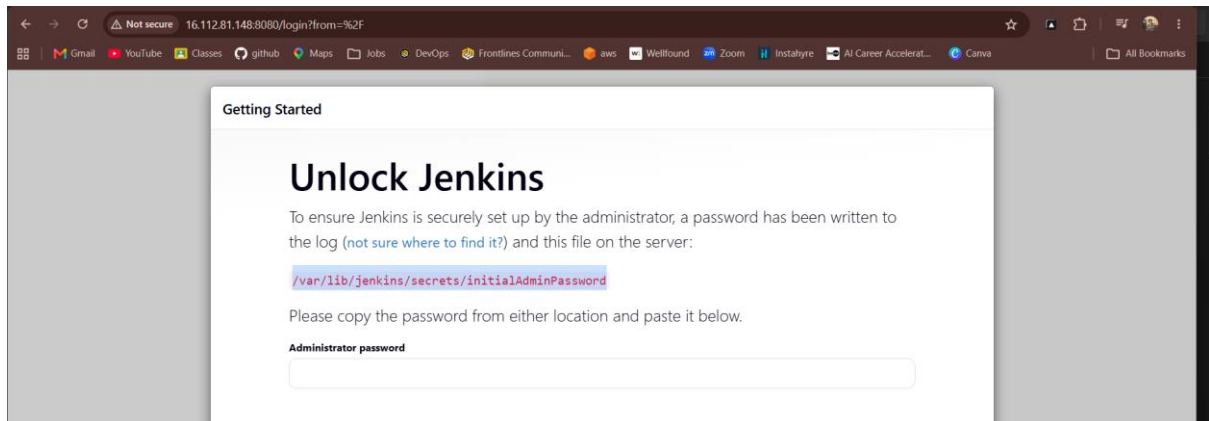
sudo systemctl enable Jenkins

sudo systemctl status Jenkins

```
[root@jenkins ~]# systemctl start jenkins
[root@jenkins ~]# systemctl enable jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /usr/lib/systemd/system/jenkins.service.
[root@jenkins ~]# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-10-03 08:35:32 UTC; 20s ago
```

Make sure to open 8080 port in Ec2 for Jenkins server → security group(Inbound rules)

http://< Ec2-Public-Ip>:8080

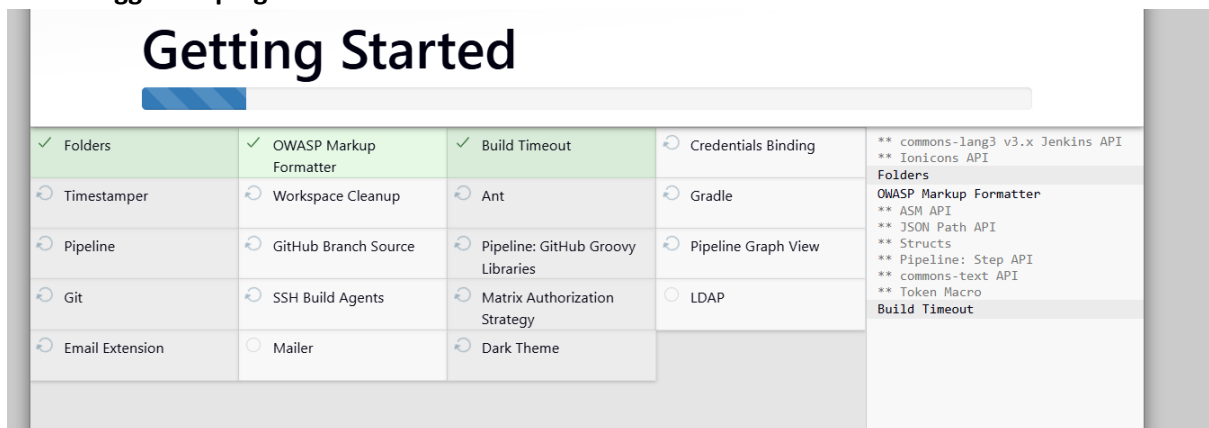


cat /var/lib/jenkins/secrets/initialAdminPassword

```
[root@jenkins ~]# cat /var/lib/jenkins/secrets/initialAdminPassword
fe0f1676e15941a8bfc8ac45ed2fdf7b
[root@jenkins ~]#
```

Paste password in JenkinsYou will land into Jenkins UI

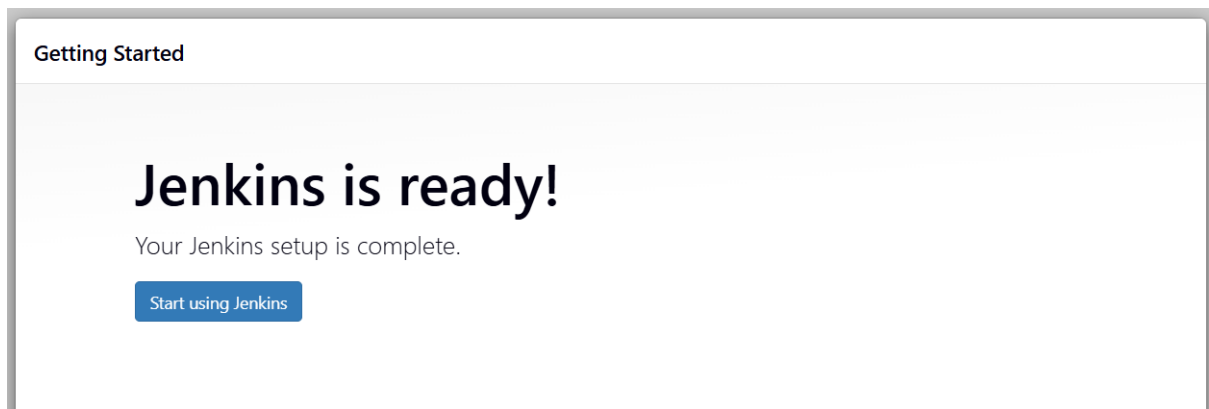
Install suggested plugins



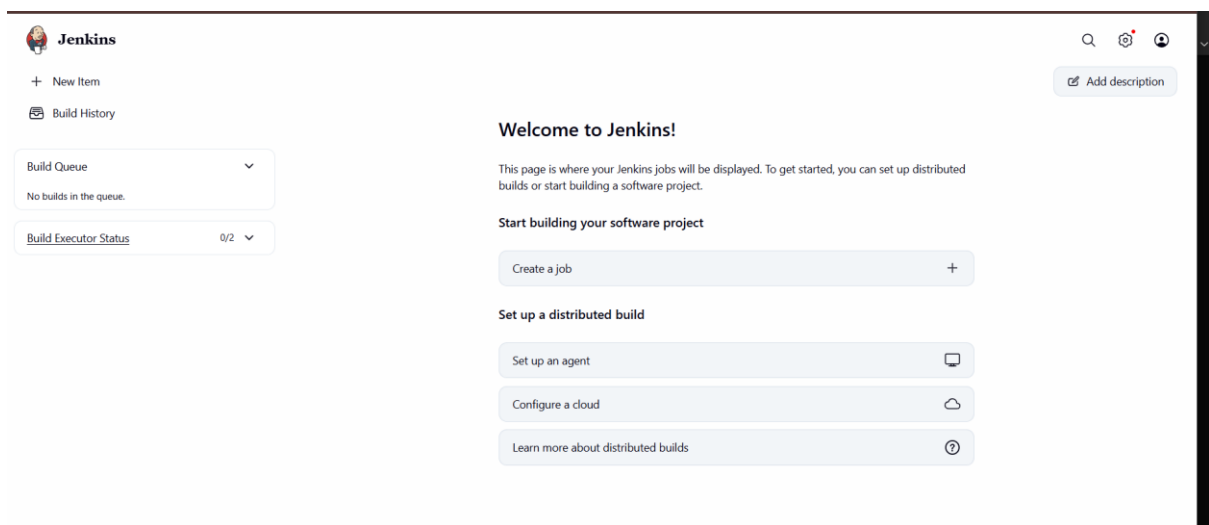
Set username and password



Start Jenkins



Welcome to Jenkins UI



Create Pipeline not free-style Job

New Item

Enter an item name

Deployment-Pipeline

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Sudo yum install git -y

```
[root@jenkins ~]# sudo yum install git -y
Last metadata expiration check: 0:23:20 ago on Sat Oct 3 08:29:45 2026.
```

git --version

git version 2.50.1

sample pipeline code

```
pipeline {
  agent any
  stages {
    stage('Hello') {
      steps {
        echo 'Hello World'
      }
    }
  }
}
```

Code Check-Out:

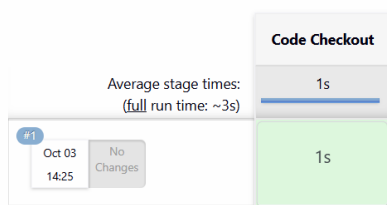
github Repo → https://github.com/DevOpsCloudZone/docker_web_stack.git

Stage1 :

```
stage (Code Checkout") {
  steps {
    git 'https://github.com/DevOpsCloudZone/docker_web_stack.git '
  }
}
```

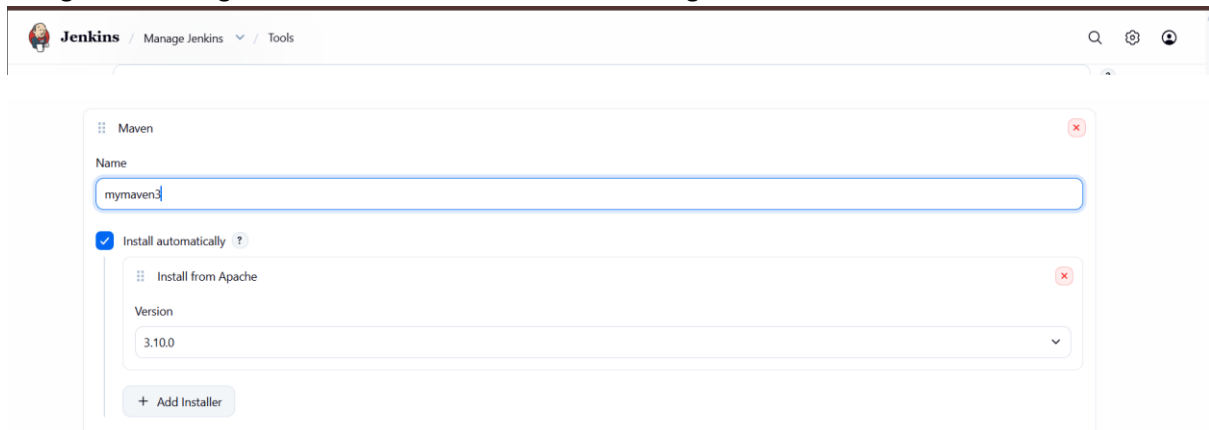
Trigger the Pipeline

Stage View



Build and Test with Maven

Navigate to Manage Jenkins >> Tools >> add Maven Configuration



Add this section under agent in pipeline

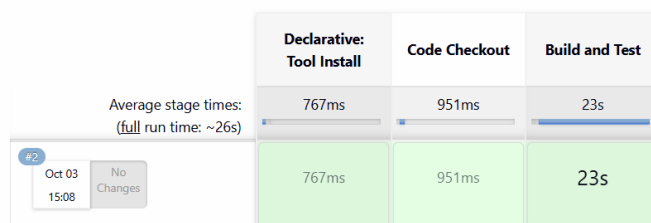
```
tools{  
    maven "mymaven3"  
}
```

Stage 2:

```
stage("Build and Test"){  
    steps{  
        sh 'mvn clean install'  
    }  
}
```

Trigger the Pipeline

Stage View



Verify the console output

Cross check the → target folder is available in Jenkins server

cd /var/lib/jenkins/workspace

cd Deployment-Pipeline ## your pipeline name here

```
[root@jenkins ~]# cd /var/lib/jenkins/workspace
[root@jenkins workspace]# ll
total 0
drwxr-xr-x. 8 jenkins jenkins 129 Oct 3 09:38 Deployment-Pipeline
drwxr-xr-x. 2 jenkins jenkins 6 Oct 3 09:38 Deployment-Pipeline@tmp
[root@jenkins workspace]# cd Deployment-Pipeline
[root@jenkins Deployment-Pipeline]# ll
total 28
drwxr-xr-x. 3 jenkins jenkins 42 Oct 3 08:55 Docker-app
drwxr-xr-x. 2 jenkins jenkins 45 Oct 3 08:55 Docker-db
-rw-r--r--. 1 jenkins jenkins 388 Oct 3 08:55 compose.yml
drwxr-xr-x. 2 jenkins jenkins 57 Oct 3 08:55 manifests
-rw-r--r--. 1 jenkins jenkins 7254 Oct 3 08:55 pom.xml
drwxr-xr-x. 4 jenkins jenkins 30 Oct 3 08:55 src
drwxr-xr-x. 11 jenkins jenkins 16384 Oct 3 09:38 target
```

Sonarqube Setup

Installation for Code Quality Analysis and Code Quality gates

Installation Steps:

New EC2 Server

Install docker

sudo yum install docker -y

```
[root@ip-172-31-37-1 ~]# docker -v
Docker version 25.0.14, build 0bab007
```

[docker -v

Docker version 25.0.14, build 0bab007

sudo systemctl start docker && systemctl enable docker && systemctl status docker

```
[root@ip-172-31-37-1 ~]# sudo systemctl start docker && systemctl enable docker && systemctl status docker
sudo: systemctl: command not found
[root@ip-172-31-37-1 ~]# sudo systemctl start docker && sudo systemctl enable docker && sudo systemctl status docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-10-03 09:50:20 UTC; 168ms ago
```

Here I am using Sonarqube Image from Dockerhub

docker pull sonarqube

verify: docker images

```
[root@ip-172-31-37-1 ~]# docker pull sonarqube
Using default tag: latest
latest: Pulling from library/sonarqube
a60ef8ec184e: Pull complete
96fe3bd0148e: Pull complete
54929e68a7af: Pull complete
4f4fb700ef54: Pull complete
0a66c41acdcb: Pull complete
065adb12f4f5: Pull complete
03217b7e6be2: Pull complete
Digest: sha256:c0f1160ccf435db4168c2d7df69af3c3ea7bfeb87395613014048fb33a68c1
Status: Downloaded newer image for sonarqube:latest
docker.io/library/sonarqube:latest
[root@ip-172-31-37-1 ~]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
sonarqube latest 01676bb7354b 31 hours ago 1.44GB
[root@ip-172-31-37-1 ~]#
```

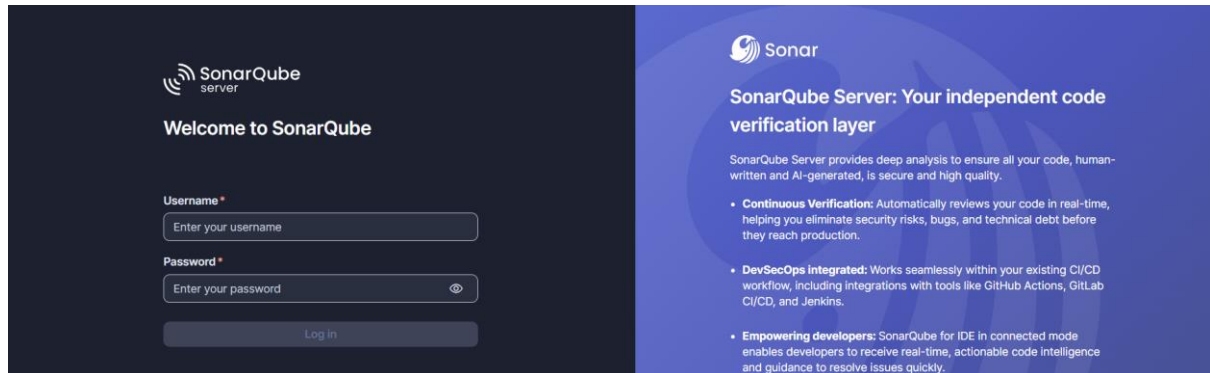
docker run -d --name sonarqube-cont -p 9000:9000 sonarqube:latest

```
[root@ip-172-31-37-1 ~]# docker run -d --name sonarqube-cont -p 9000:9000 sonarqube:latest
e274093ab14b9183ce86089e00e89a42c65b8284ef7274653cb1ce4f71184e9
```

Access the SonarQube application

Make sure port 9000 is allowed in the SonarQube server's Security Group under Inbound Rules

http://<EC2-pubilc-IP>:9000



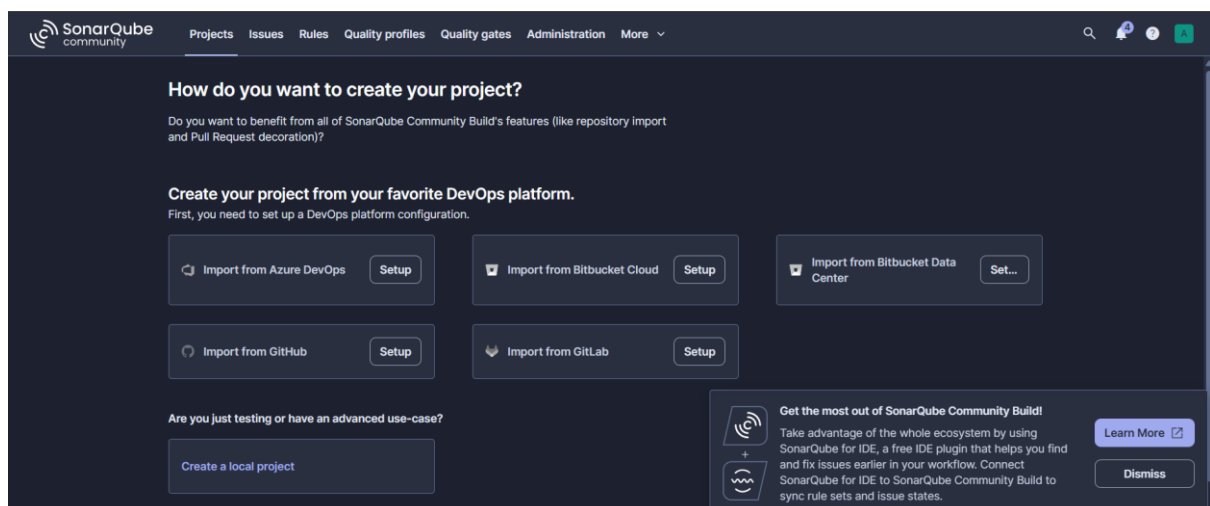
By default

username: admin

Password: admin

After this update to new password

Sonarqube UI will be displayed as shown below.



Click on → create a local project



Enter values → your own project name

Create a local project

Project display name * ⓘ
sonarproject

Project key * ⓘ
sonarproject

Main branch name *
main

The name of your project's default branch [Learn More](#) ⓘ

Choose values like this >>>>>> click on Create Project

2 of 2

Set up new code for project

The new code definition sets the criteria for what's considered new code, allowing you to focus on the most recent changes in your project.

This setting allows project administrators to customize the new code definition for their project. [Learn more in documentation](#) ⓘ

☐ Follows the instance's default
Current default: Previous version

☒ Is custom
Choose whether it should be the previous version, a number of days or a reference branch

☐ Previous version
Any code that has changed since the previous version is considered new code. Recommended for projects following regular versions or releases.

☒ Number of days
Any code that has changed in the last x days is considered new code. If no action is taken on a new issue after x days, this issue will become part of the overall code. Recommended for projects following continuous delivery.

Define the number of days *
30

☐ Reference branch
Choose a reference branch as the baseline for the new code. Recommended for projects using feature branches.

The next UI will be displayed as shown below.

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More ▾

sonarproject / Analysis Method

Analysis Method

Use this page to manage and set-up the way your analyses are performed.

How do you want to analyze your repository?

☐ With Jenkins

☐ With GitHub Actions

☒ With Bitbucket Pipelines

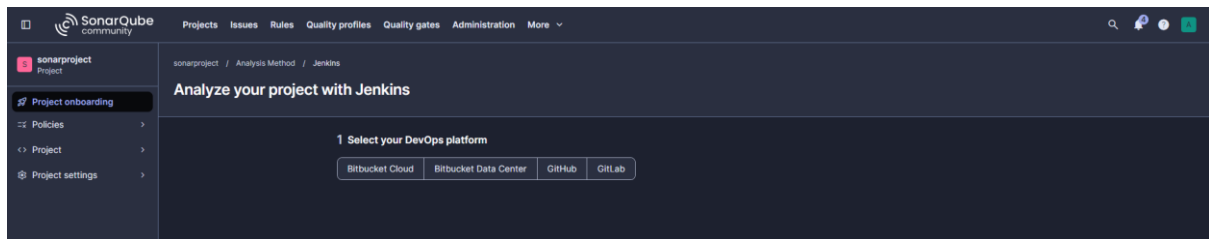
☐ With GitLab CI

☐ With Azure Pipelines

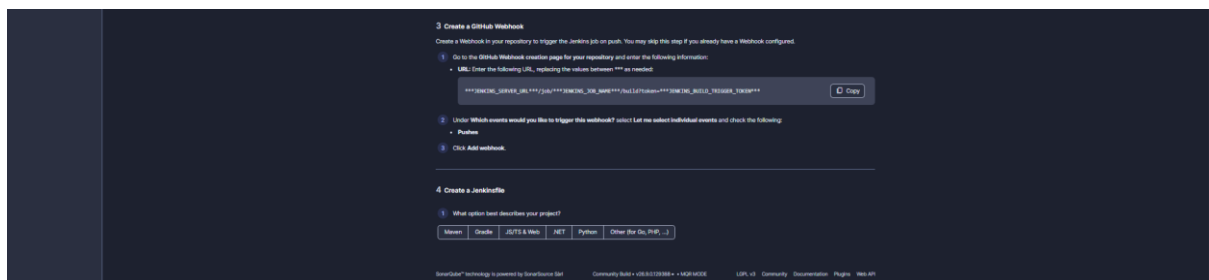
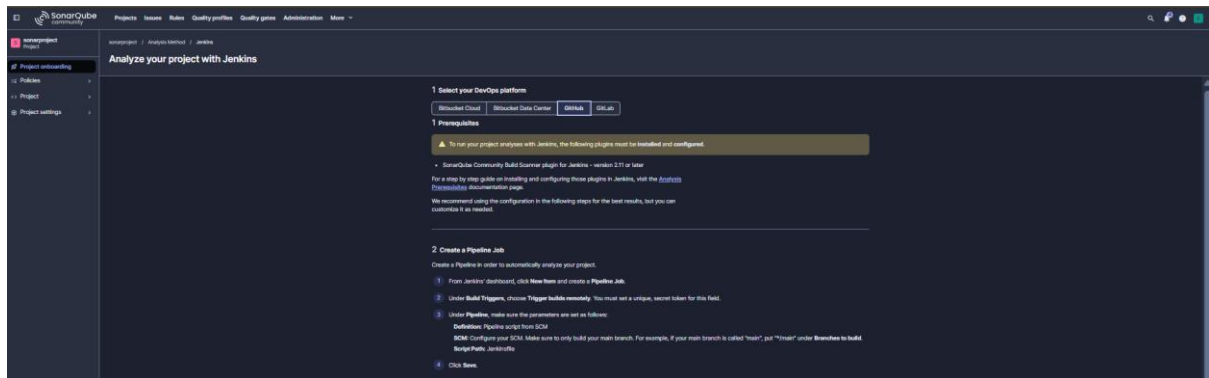
Other CI
SonarQube Community Build integrates with your workflow no matter which CI tool you're using.

Locally
Use this for testing or advanced use-case. Other modes are recommended to help you set up your CI environment.

Select **with Jenkins** >> next UI



Select **GitHub**



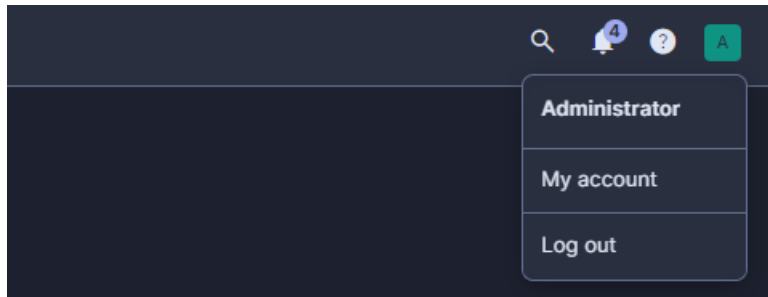
Select **maven**



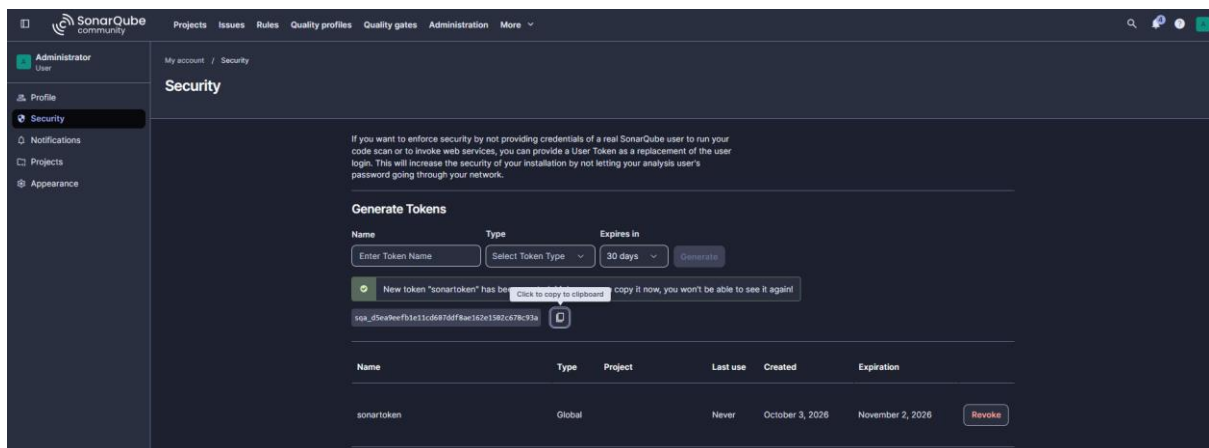
Copy that code somewhere for → In Jenkins Pipeline we will need this

We need to create Token in sonarqube for accessing sonarqube through Jenkins

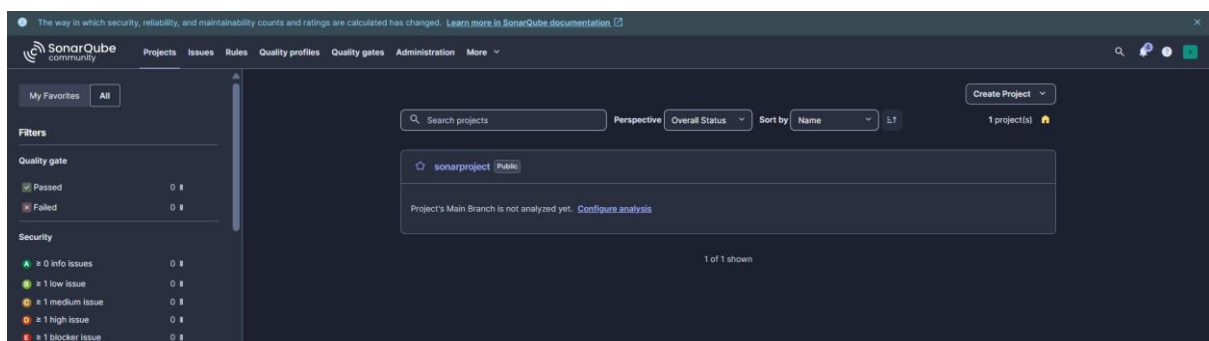
Click on My Profile Icon/Account and select **My account**



Click security >> select values as shown in below screen >> you will get token >> save it

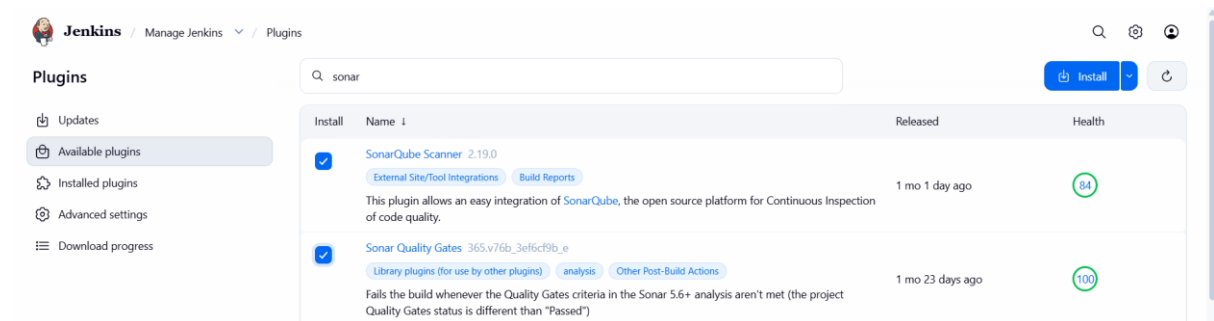


Click on Projects you will you Your Project here



In Jenkins

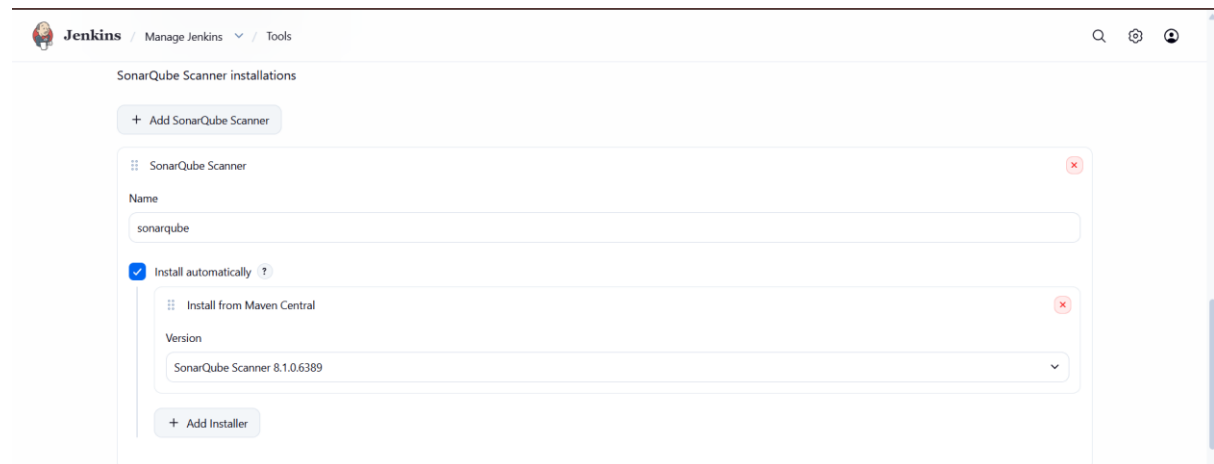
Download SonarQube Scanner and Sonar Quality Gates in plugins



The screenshot shows the Jenkins 'Manage Jenkins' > 'Plugins' page. A search bar at the top contains 'sonar'. Below the search bar, a table lists installed plugins. The 'SonarQube Scanner' plugin (version 2.19.0) is installed and has a health status of 84. The 'Sonar Quality Gates' plugin (version 365.v76b_3ef6cf9b_e) is also installed and has a health status of 100. The left sidebar shows the 'Available plugins' tab selected.

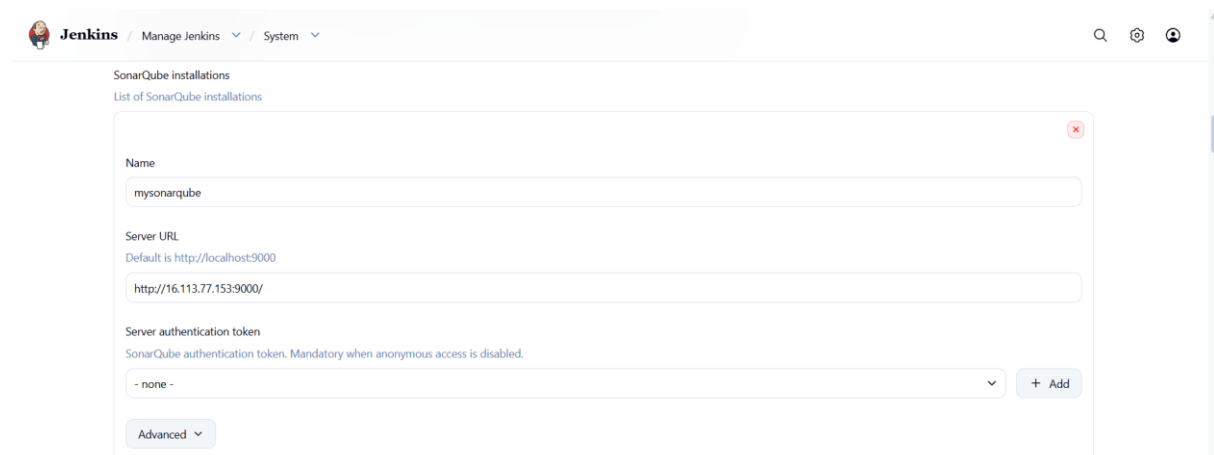
Install	Name	Released	Health
☒	SonarQube Scanner 2.19.0 External Site/Tool Integrations Build Reports This plugin allows an easy integration of SonarQube , the open source platform for Continuous Inspection of code quality.	1 mo 1 day ago	84
☒	Sonar Quality Gates 365.v76b_3ef6cf9b_e Library plugins (for use by other plugins) analysis Other Post-Build Actions Fails the build whenever the Quality Gates criteria in the Sonar 5.6+ analysis aren't met (the project Quality Gates status is different than "Passed")	1 mo 23 days ago	100

Navigate to Manage Jenkins >> Tools >> SonarQube Scanner Installations



The screenshot shows the Jenkins 'Manage Jenkins' > 'Tools' > 'SonarQube Scanner installations' page. A button '+ Add SonarQube Scanner' is at the top. Below it, a configuration card for 'SonarQube Scanner' is shown. The 'Name' field is 'sonarqube'. The 'Install automatically' checkbox is checked. Under 'Install from Maven Central', the 'Version' is set to 'SonarQube Scanner 8.1.0.6389'. An '+ Add Installer' button is at the bottom of the card.

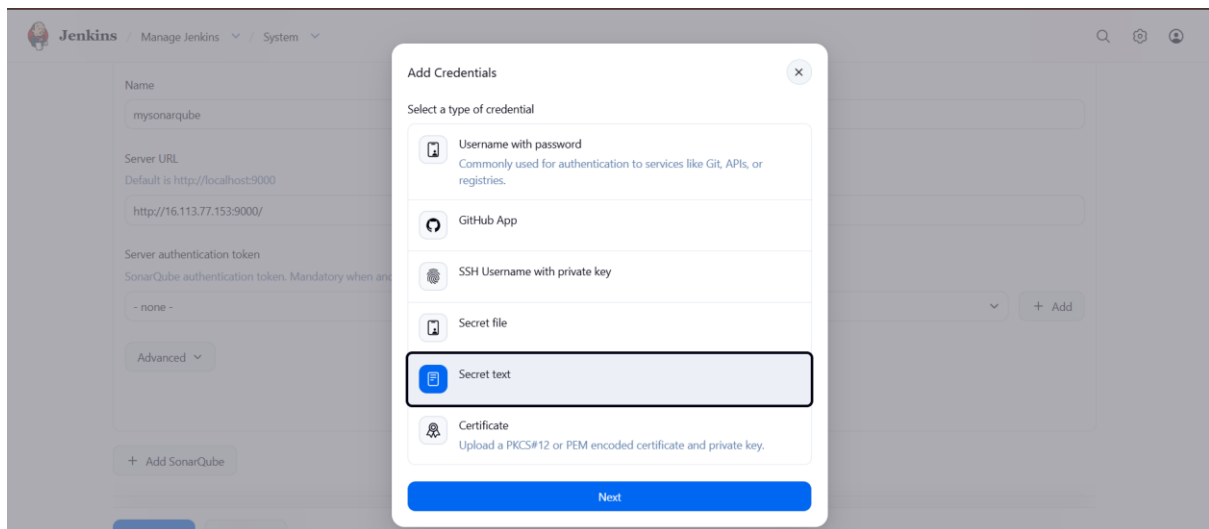
Navigate to Manage Jenkins >> System >> SonarQube Installations



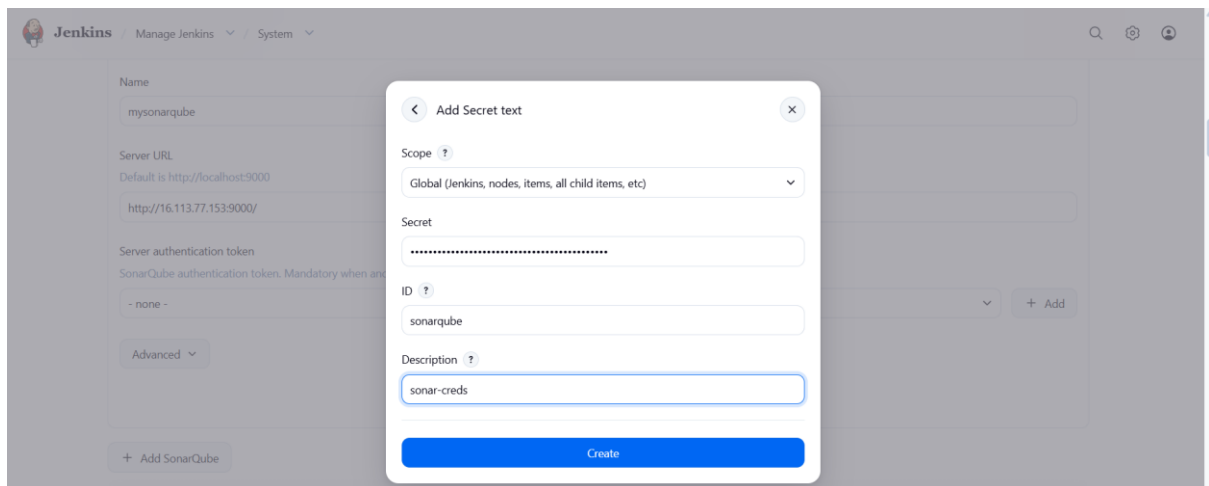
The screenshot shows the Jenkins 'Manage Jenkins' > 'System' > 'SonarQube installations' page. The title is 'List of SonarQube installations'. A configuration card for 'mysonarqube' is shown. The 'Name' field is 'mysonarqube'. The 'Server URL' field is 'http://16.113.77.153:9000/'. The 'Server authentication token' field is set to '- none -'. An '+ Add' button is next to the token field. An 'Advanced' dropdown menu is at the bottom of the card.

Sonar authentication token add token here

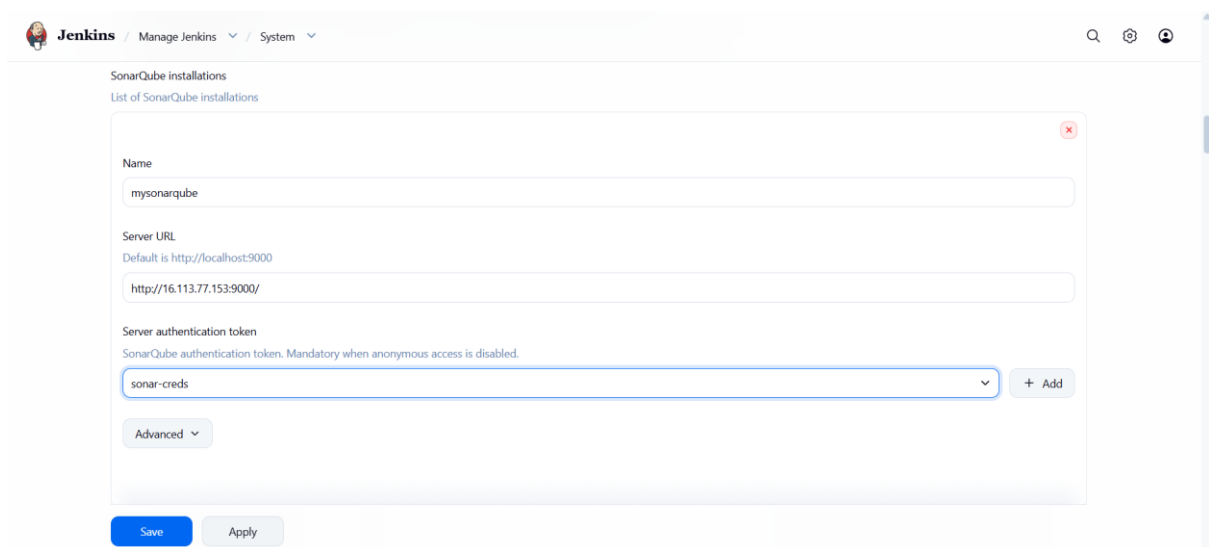
Select → Secret text



Copy our generated token here → → which we generated in sonarqube



Save and select credentials



Stage 3: Code Quality Analysis

```
stage("Code Quality Analysis"){
    steps{
        withSonarQubeEnv('mysonarqube') {
            // Here copy code from Sonarqube
            sh "mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar -
Dsonar.projectKey=sonarproject -Dsonar.projectName='sonarproject'"
        }
    }
}
```

Maven Tool Configuration:

Maven is configured in the Jenkins Declarative Pipeline using tools { maven "mymaven3" }. Jenkins automatically makes the configured Maven installation available on the system PATH, so the pipeline can directly use the mvn command without defining def mvn = tool 'mymaven3'.

SonarQube Analysis with Maven:

The pipeline runs SonarQube analysis using the Maven SonarQube Scanner plugin:

```
mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar
```

Here, Maven executes the build and verification phases and then invokes the SonarQube Maven plugin to analyze the source code and send the analysis results to the configured SonarQube server.

SonarQube Project Properties / GAV:

The SonarQube project is identified using properties such as:

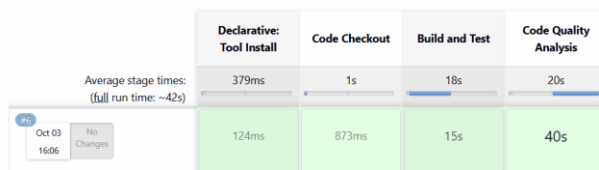
```
-Dsonar.projectKey=sonarproject
```

```
-Dsonar.projectName=sonarproject
```

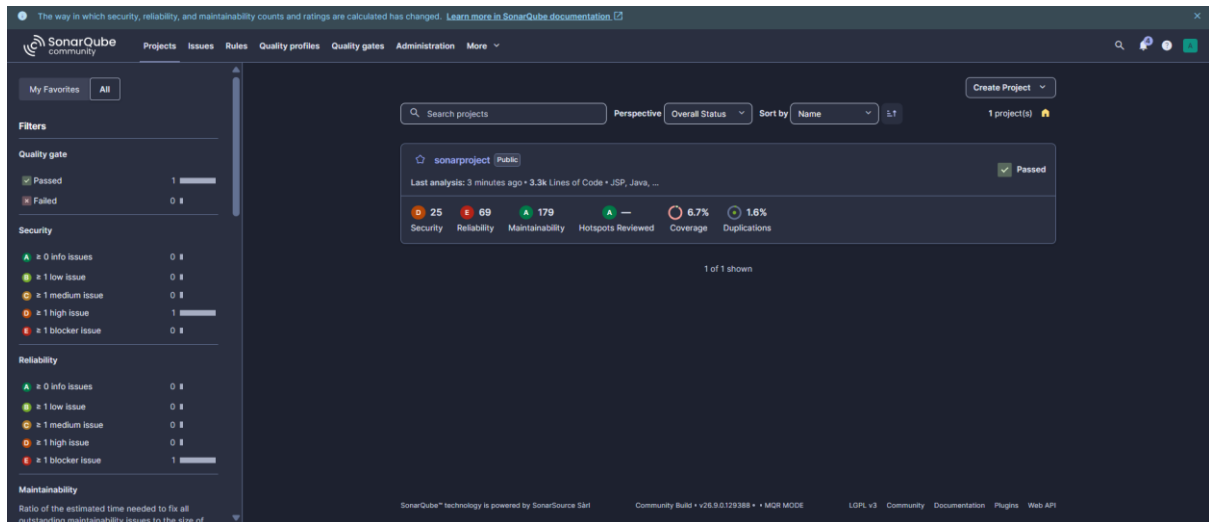
These SonarQube properties identify the project in SonarQube, while Maven GAV coordinates (groupId, artifactId, version) identify the Maven artifact, for example com.visualpathit:vprofile:v2. They are separate concepts: GAV identifies the build artifact; SonarQube project properties identify the code-quality project

Trigger the Pipeline

Deployment-Pipeline - Stage View



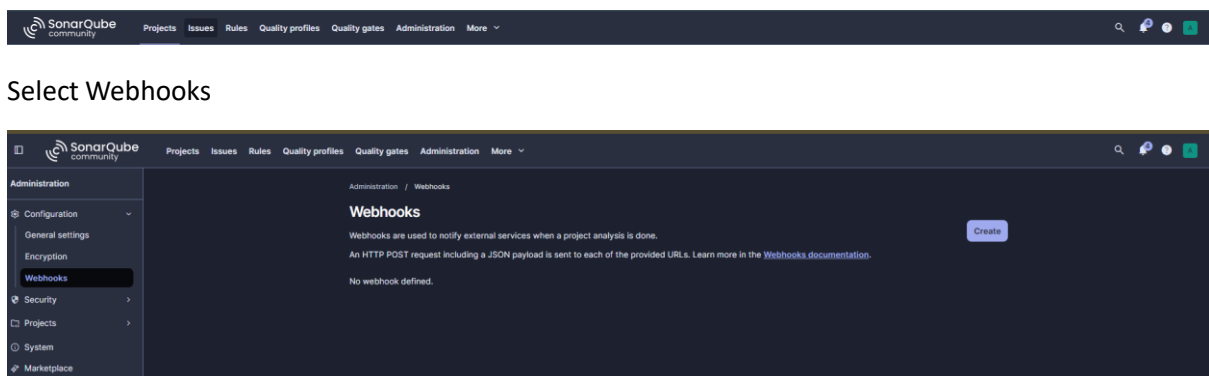
Verify the analysis results in the SonarQube UI



Code Quality Gates

For Quality Gates >> Configure Webhooks in SonarQube UI

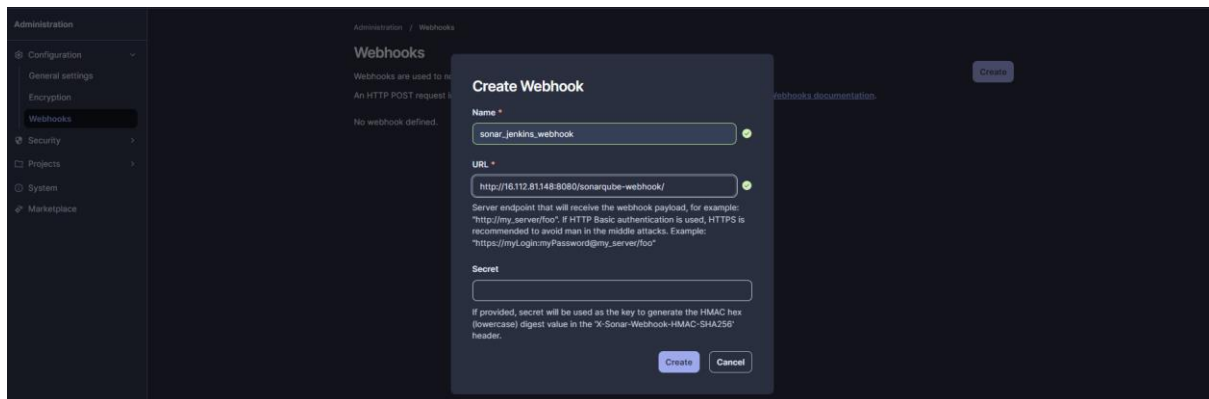
Click on Administration



Click on Create

URL: Jenkins URL

<http://16.112.81.148:8080/sonarqube-webhook/>



The screenshot shows the 'Create Webhook' dialog box in SonarQube. The dialog has a dark theme. It contains the following fields and text:

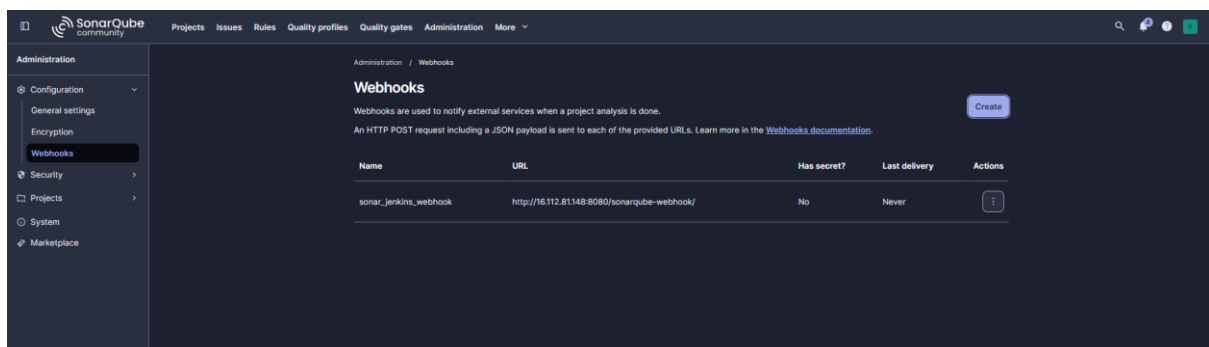
- Name ***: sonar_jenkins_webhook
- URL ***: http://16.112.81.148:8080/sonarqube-webhook/
- Secret**: (empty field)

Below the fields, there is explanatory text: "Server endpoint that will receive the webhook payload, for example: 'http://my_server/foo'. If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example: 'https://myLogin:myPassword@my_server/foo'". Below this text, it says: "If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header."

At the bottom right of the dialog are two buttons: "Create" and "Cancel".

create

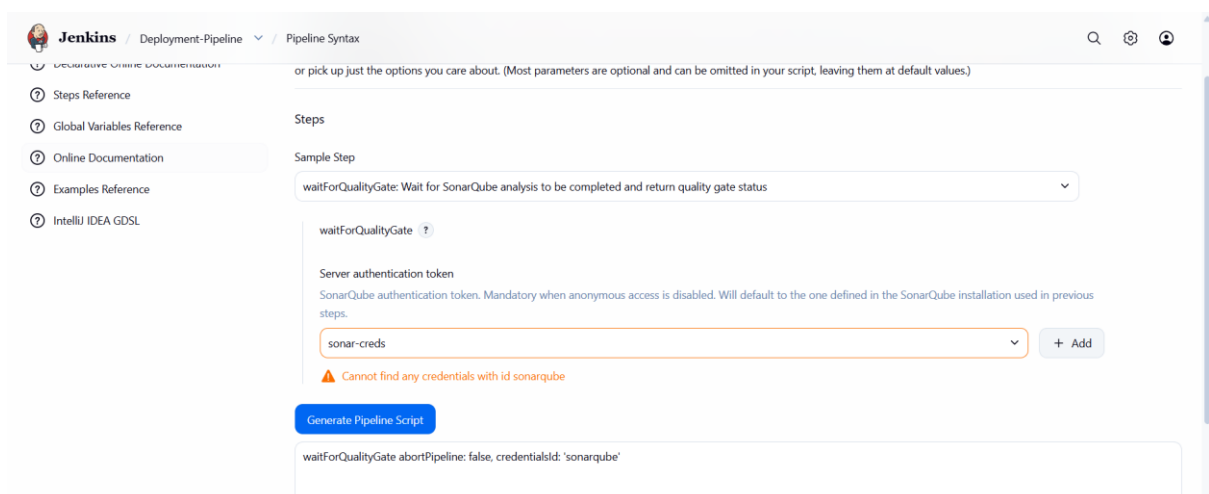
Webhook created in sonarqube



The screenshot shows the 'Webhooks' page in SonarQube. The page has a dark theme. At the top, there is a 'Create' button. Below it, there is a table with the following columns: Name, URL, Has secret?, Last delivery, and Actions.

Name	URL	Has secret?	Last delivery	Actions
sonar_jenkins_webhook	http://16.112.81.148:8080/sonarqube-webhook/	No	Never	[More options icon]

In pipeline syntax we got pipeline script



The screenshot shows the 'Pipeline Syntax' page in Jenkins. The page has a light theme. It displays the 'waitForQualityGate' step in a pipeline script. The step is defined as:

```
waitForQualityGate {  
    serverAuthenticationToken = 'sonar-creds'  
}
```

Below the script, there is a 'Generate Pipeline Script' button. The generated script is shown below the button:

```
waitForQualityGate {  
    abortPipeline = false  
    credentialsId = 'sonarqube'  
}
```

Stage Syntax:

```
stage('Code Quality Gates'){
    steps{
        waitForQualityGate abortPipeline: true, credentialsId: 'sonarqube'
    }
}
```

Trigger the Pipeline

Declarative: Tool Install	Code Checkout	Build and Test	Code Quality Analysis	Code Quality Gates
73ms	805ms	14s	30s	158ms
46ms	794ms	13s	28s	88ms (paused for 2s)

Nexus Server Installation

New EC2-Server for Nexus Installation

sudo yum install java-21-amazon-corretto -y

```
[root@ip-172-31-34-223 opt]# java --version
openjdk 21.0.12.1 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
```

cd /opt

wget https://download.sonatype.com/nexus/3/nexus-3.96.4-01-linux-x86_64.tar.gz

```
[root@ip-172-31-34-223 ~]# cd /opt
[root@ip-172-31-34-223 opt]# wget https://download.sonatype.com/nexus/3/nexus-3.96.4-01-linux-x86_64.tar.gz
--2026-10-03 11:09:51-- https://download.sonatype.com/nexus/3/nexus-3.96.4-01-linux-x86_64.tar.gz
Resolving download.sonatype.com (download.sonatype.com)... 104.18.0.134, 104.18.1.134, 2606:4700::6812:86, ...
Connecting to download.sonatype.com (download.sonatype.com)|104.18.0.134|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 517500940 (494M) [application/x-tar]
Saving to: 'nexus-3.96.4-01-linux-x86_64.tar.gz'

nexus-3.96.4-01-linux-x86_64.tar.gz 100%[=====>] 493.53M 56.5MB/s in 9.2s

2026-10-03 11:06:01 (53.6 MB/s) - 'nexus-3.96.4-01-linux-x86_64.tar.gz' saved [517500940/517500940]

[root@ip-172-31-34-223 opt]# ll
total 505376
drwxr-xr-x. 4 root root 33 Sep 28 23:38 aws
-rw-r--r--. 1 root root 517500940 Oct 3 11:06 nexus-3.96.4-01-linux-x86_64.tar.gz
[root@ip-172-31-34-223 opt]#
```

tar -zxvf nexus-3.96.4-01-linux-x86_64.tar.gz

after this command we will get this

```
[root@ip-172-31-34-223 opt]# ll
total 505376
drwxr-xr-x. 4 root root      33 Sep 28 23:38 aws
drwxr-xr-x. 6 root root      94 Oct  3 11:06 nexus-3.96.4-01
-rw-r--r--. 1 root root 517500940 Oct  3 11:06 nexus-3.96.4-01-linux-x86_64.tar.gz
drwxr-xr-x. 3 root root      20 Sep 29 17:33 sonatype-work
[root@ip-172-31-34-223 opt]#
```

useradd nexus

chown -R nexus:nexus nexus-3.96.4-01 sonatype-work/

```
[root@ip-172-31-34-223 opt]# useradd nexus
useradd: user 'nexus' already exists
[root@ip-172-31-34-223 opt]# chown -R nexus:nexus nexus-3.96.4-01 sonatype-work/
[root@ip-172-31-34-223 opt]#
```

su - nexus

user changed

```
[root@ip-172-31-34-223 opt]# su - nexus
[nexus@ip-172-31-34-223 ~]$
```

cd /opt/nexus-3.96.4-01/bin/

```
[nexus@ip-172-31-34-223 ~]$ cd /opt
[nexus@ip-172-31-34-223 opt]$ ll
total 505376
drwxr-xr-x. 4 root root      33 Sep 28 23:38 aws
drwxr-xr-x. 6 nexus nexus    94 Oct  3 11:06 nexus-3.96.4-01
-rw-r--r--. 1 root root 517500940 Oct  3 11:06 nexus-3.96.4-01-linux-x86_64.tar.gz
drwxr-xr-x. 3 nexus nexus    20 Sep 29 17:33 sonatype-work
[nexus@ip-172-31-34-223 opt]$
```

```
[nexus@ip-172-31-34-223 opt]$ ll
total 505376
drwxr-xr-x. 4 root root      33 Sep 28 23:38 aws
drwxr-xr-x. 6 nexus nexus    94 Oct  3 11:06 nexus-3.96.4-01
-rw-r--r--. 1 root root 517500940 Oct  3 11:06 nexus-3.96.4-01-linux-x86_64.tar.gz
drwxr-xr-x. 3 nexus nexus    20 Sep 29 17:33 sonatype-work
[nexus@ip-172-31-34-223 opt]$ cd nexus-3.96.4-01/
[nexus@ip-172-31-34-223 nexus-3.96.4-01]$ ll
total 16
-rw-r--r--. 1 nexus nexus 1022 Sep 29 17:20 NOTICE.txt
-rw-r--r--. 1 nexus nexus 11249 Sep 29 17:20 OSS-LICENSE.txt
drwxr-xr-x. 2 nexus nexus   89 Oct  3 11:06 bin
drwxr-xr-x. 2 nexus nexus  26 Oct  3 11:06 deploy
drwxr-xr-x. 7 nexus nexus 143 Oct  3 11:06 etc
drwxr-xr-x. 3 nexus nexus  45 Sep 29 17:33 jdk
[nexus@ip-172-31-34-223 nexus-3.96.4-01]$ cd bin
[nexus@ip-172-31-34-223 bin]$ ll
total 327772
-rwxr-xr-x. 1 nexus nexus   6735 Sep 29 16:36 nexus
-rw-r--r--. 1 nexus nexus   1325 Sep 29 16:36 nexus.vmoptions
-rw-r--r--. 1 nexus nexus 335623966 Sep 29 17:31 sonatype-nexus-repository-3.96.4-01.jar
```

./nexus start

```
[nexus@ip-172-31-34-223 bin]$ ./nexus start
Starting nexus
[nexus@ip-172-31-34-223 bin]$
```

Full commands:

```
cd /opt
```

```
wget https://download.sonatype.com/nexus/3/nexus-3.96.0-09-linux-x86_64.tar.gz
```

```
tar -zxvf nexus-3.96.0-09-linux-x86_64.tar.gz
```

```
useradd nexus
```

```
chown -R nexus:nexus nexus-3.96.0-09 sonatype-work
```

```
su - nexus
```

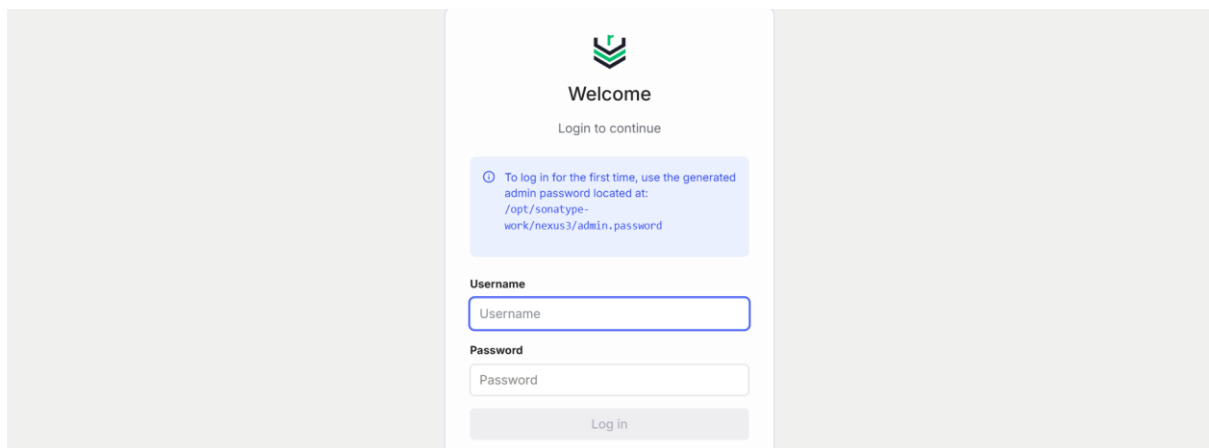
```
cd /opt/nexus-3.96.0-09/bin/
```

```
./nexus start
```

Access the application with Nexus Public Ip Address

Make sure port 8081 is open in the EC2 Security Group.

<http://<EC2-Public-IP>:8081>

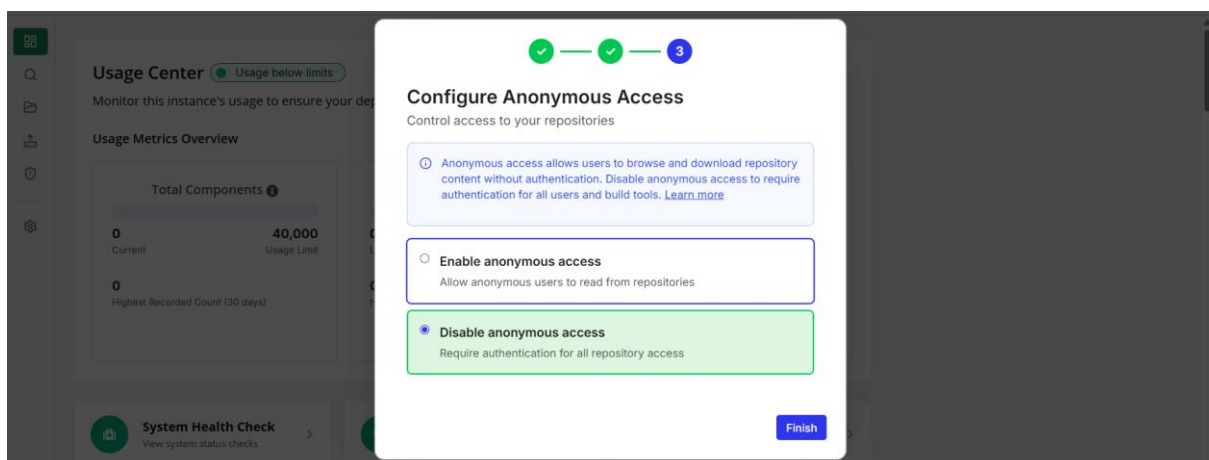
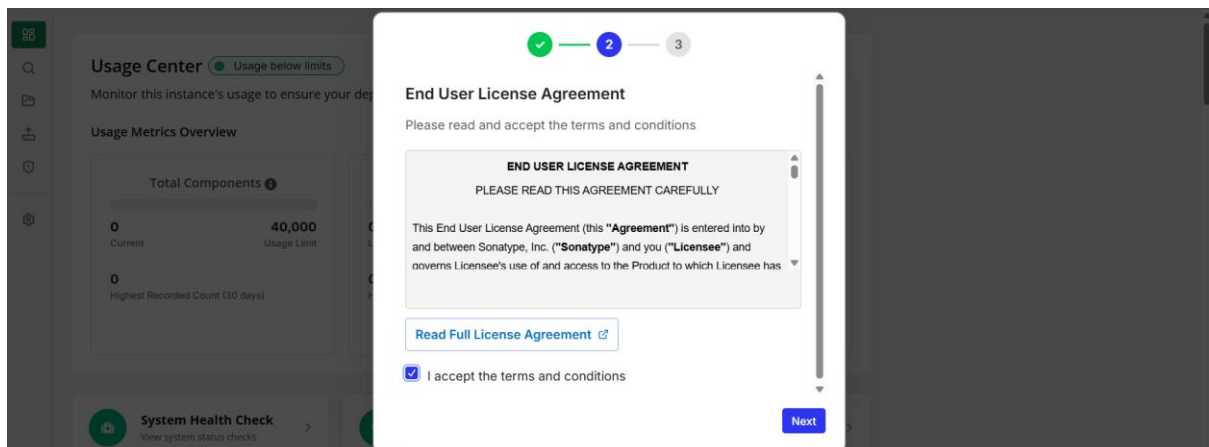
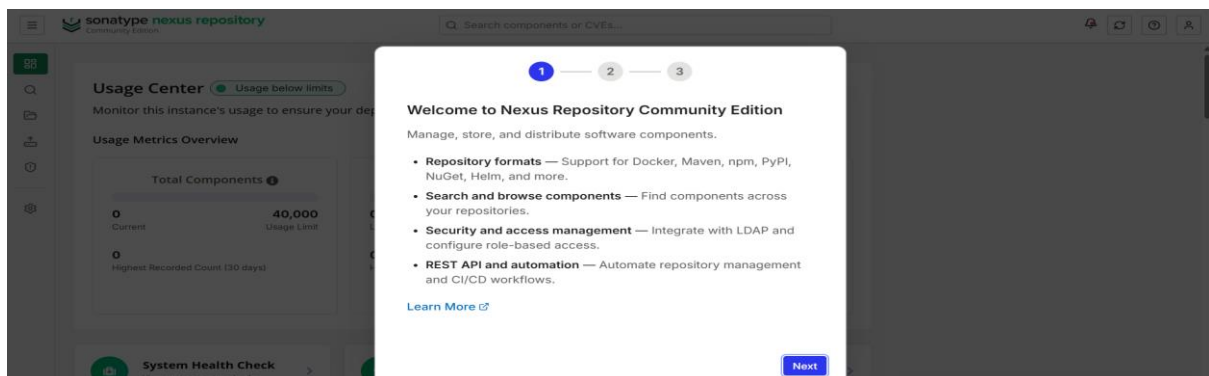
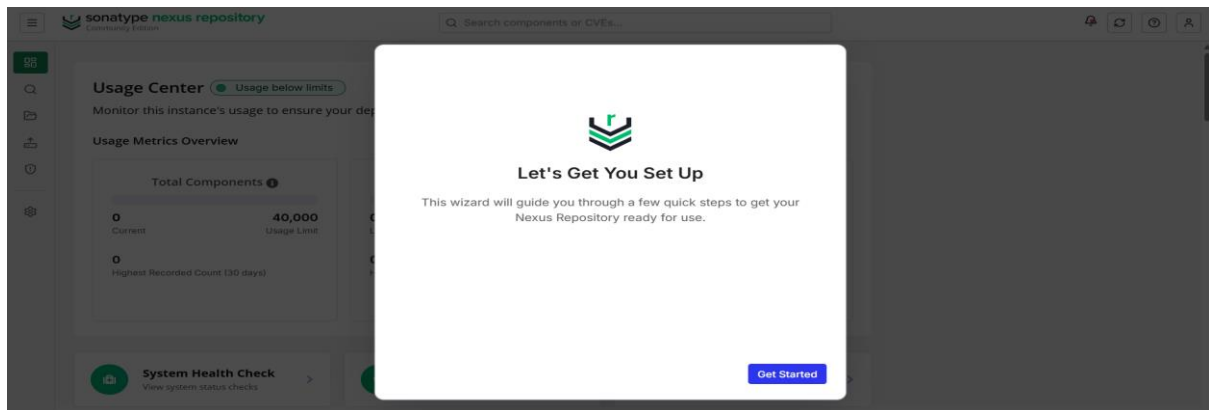
The image shows the Nexus 3 web interface. At the top is the Nexus logo (a green 'N' with a checkmark) and the word 'Welcome'. Below that is the text 'Login to continue'. A blue information box contains a note: 'To log in for the first time, use the generated admin password located at: /opt/sonatype-work/nexus3/admin.password'. Below this are two input fields: 'Username' with the placeholder text 'Username' and 'Password' with the placeholder text 'Password'. At the bottom is a 'Log in' button.

Enter both Values

Username: **admin**

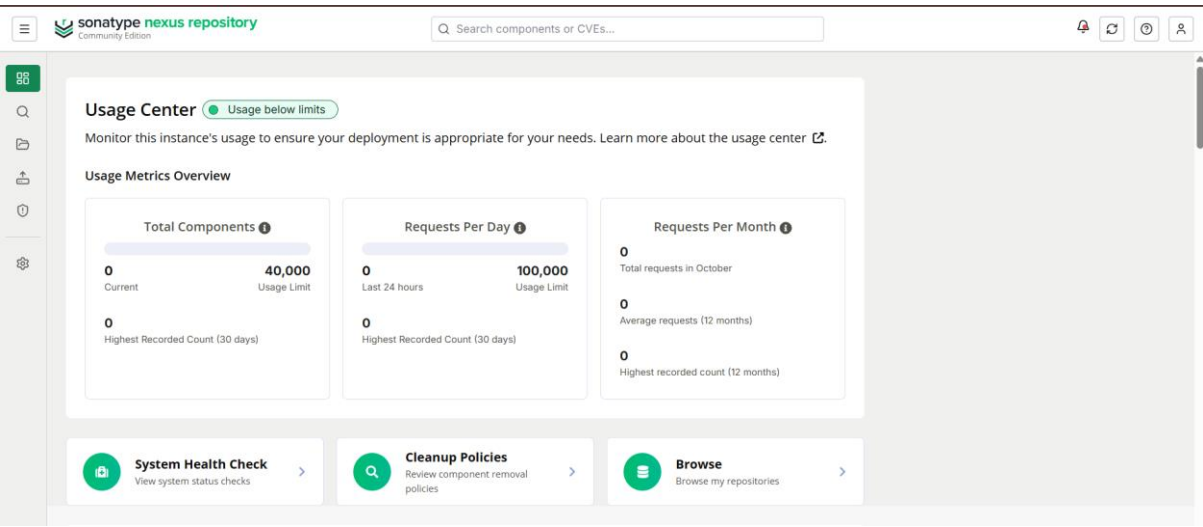
Password : **cat /opt/sonatype-work/nexus3/admin.password**

```
[nexus@ip-172-31-34-223 bin]$ cat /opt/sonatype-work/nexus3/admin.password
d278baf6-6c4b-4b54-bde1-748ce71376b5[nexus@ip-172-31-34-223 bin]$
```

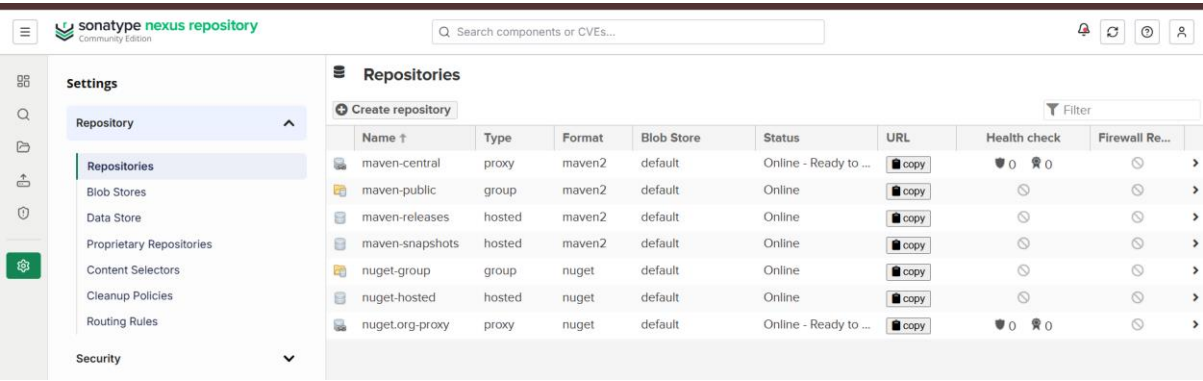


All steps are completed

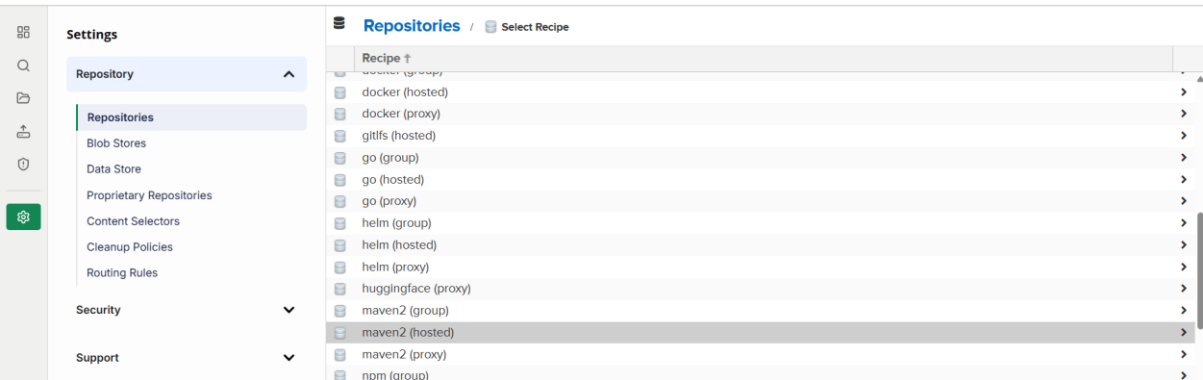
Nexus UI



Navigate to Settings >> Repositories



Create Repository -> Select maven(hosted)



Name : nexusrepo and Allow redeploy

Settings

Repository

Repositories

Blob Stores

Data Store

Proprietary Repositories

Content Selectors

Cleanup Policies

Routing Rules

Security

Support

System

Repositories / Select Recipe / Create Repository: nuget (hosted)

Name:

A unique identifier for this repository

nexusrepo

Online:

☒ If checked, the repository accepts incoming requests

Storage

Blob store:

Blob store used to store repository contents

default

Strict Content Type Validation:

☒ Validate that all content uploaded to this repository is of a MIME type appropriate for the repository format

Hosted

Deployment policy:

Controls if deployments of and updates to artifacts are allowed

Allow redeploy

Proprietary Components:

☐ Components in this repository count as proprietary for namespace conflict attacks (requires Sonatype Nexus Firewall)

Our nexusrepo created

Settings

Repository

Repositories

Blob Stores

Data Store

Proprietary Repositories

Content Selectors

Cleanup Policies

Routing Rules

Security

Repositories

Create repository

Filter

Name ↑	Type	Format	Blob Store	Status	URL	Health check	Firewall Re...
maven-central	proxy	maven2	default	Online - Ready to ...		0 0	
maven-public	group	maven2	default	Online			
maven-releases	hosted	maven2	default	Online			
maven-snapshots	hosted	maven2	default	Online			
nexusrepo	hosted	nuget	default	Online			
nuget-group	group	nuget	default	Online			
nuget-hosted	hosted	nuget	default	Online			
nuget.org-proxy	proxy	nuget	default	Online - Ready to ...		0 0	

Click on Browse

sonatype nexus repository

Community Edition

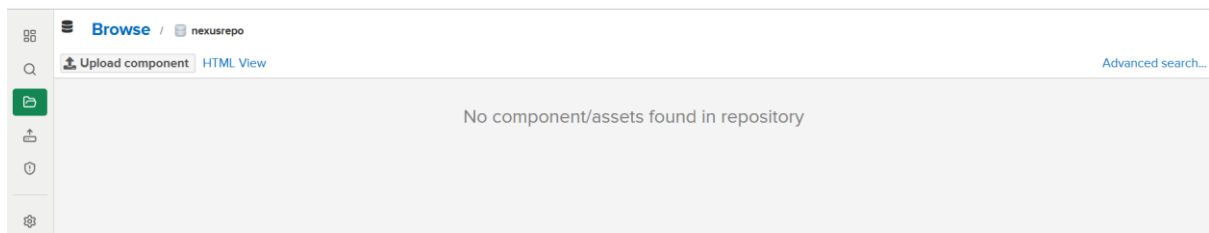
Search components or CVEs...

Browse

Filter

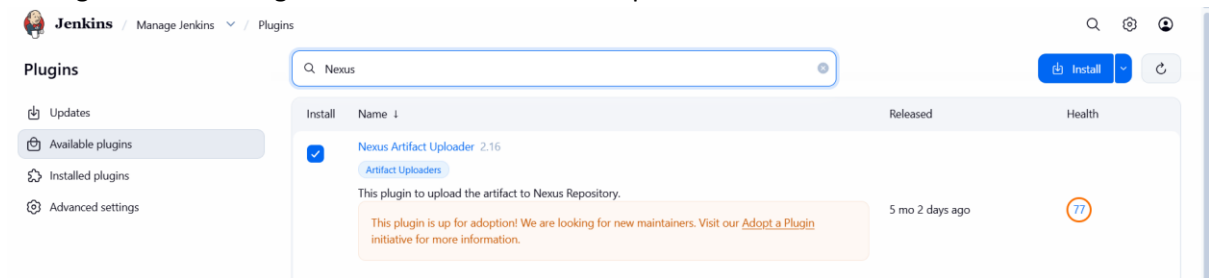
Name ↑	Type	Format	Status	URL	Health check	Firewall Re...
maven-central	proxy	maven2	Online - Ready to Connect		0 0	
maven-public	group	maven2	Online			
maven-releases	hosted	maven2	Online			
maven-snapshots	hosted	maven2	Online			
nexusrepo	hosted	nuget	Online			
nuget-group	group	nuget	Online			
nuget-hosted	hosted	nuget	Online			
nuget.org-proxy	proxy	nuget	Online - Ready to Connect		0 0	

Click on our repo →→→ you will found nothing

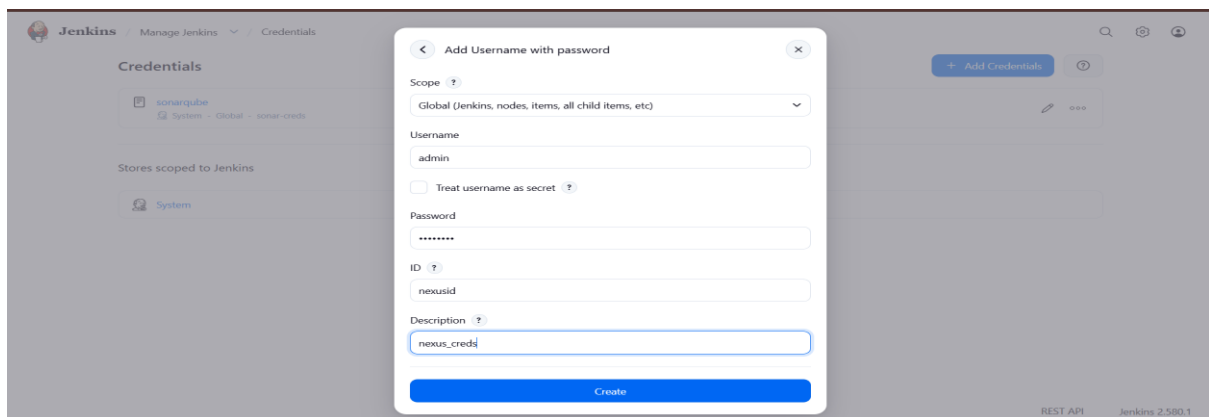


Jump on to Jenkins Server

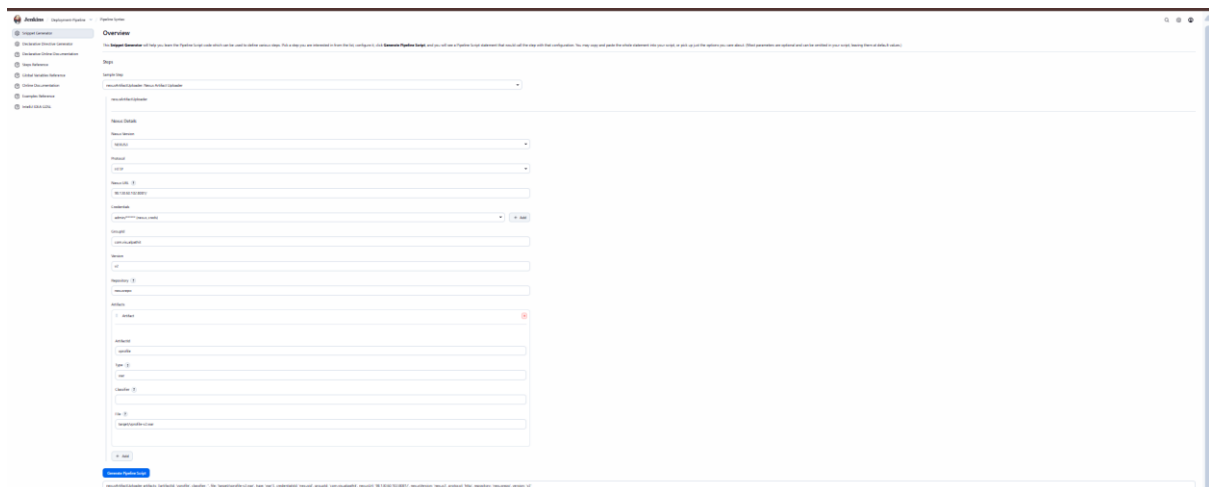
Manage Jenkins → Plugins → Install Nexus Artifact Uploader



Add nexus Credentials in → → → Navigate to Manage Jenkins >> Credentials >> Add Credentials



Generate syntax by using pipeline syntax method by updating all values



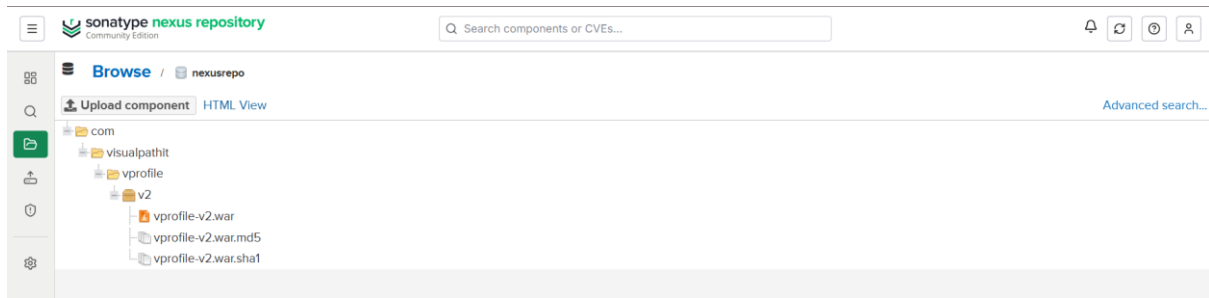
Stage : Artifact Uploader (Pipeline Syntax)

```
stage('Artifact') {  
  steps {  
    nexusArtifactUploader(  
      artifacts: [[  
        artifactId: 'vprofile',  
        classifier: '',  
        file: 'target/vprofile-v2.war',  
        type: 'war'  
      ]],  
      credentialsId: 'nexusid',  
      groupId: 'com.visualpathit',  
      nexusUrl: '98.130.60.102:8081',  
      nexusVersion: 'nexus3',  
      protocol: 'http',  
      repository: 'nexusrepo',  
      version: 'v2'  
    )  
  }  
}
```

Trigger the Pipeline

	Declarative: Tool Install	Code Checkout	Build and Test	Code Quality Analysis	Code Quality Gates	Artifact Uploader
Average stage times: (full run time: ~48s)	77ms	807ms	14s	31s	170ms	379ms
No Changes	80ms	780ms	13s	29s	147ms (paused for 1s)	797ms

After the pipeline completes, verify the artifact in the Nexus repository by navigating to Browse → nexusrepo.



Next Create Images from Docker files: In Jenkins Server

Install docker in Jenkins server

sudo yum install docker -y

```
[root@jenkins ~]# docker -v
Docker version 25.0.14, build 0bab007
```

sudo systemctl start docker && systemctl enable docker && systemctl status docker

```
[root@jenkins ~]# sudo systemctl start docker && systemctl enable docker && systemctl status docker
sudo: systemctl: command not found
[root@jenkins ~]# sudo systemctl start docker && systemctl enable docker && systemctl status docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-10-03 18:14:42 UTC; 157ms ago
```

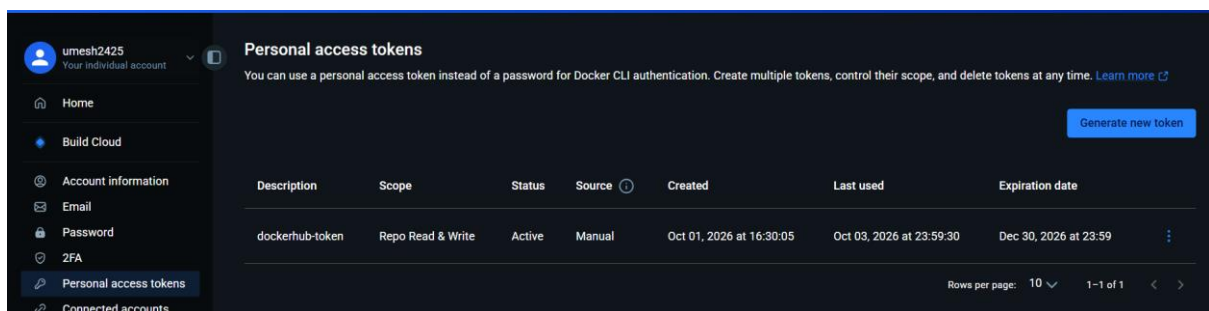
Login to DockerHub

Manual Checking in Jenkins Server

username : umesh2425 #dockerhub username

password : dockerhub token

click on profile >> Account settings >> personal access tokens



Docker login

```
[root@jenkins Deployment-Pipeline]# docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is required for organizations using SSO. Learn more at https://docs.docker.com/go/access-tokens/

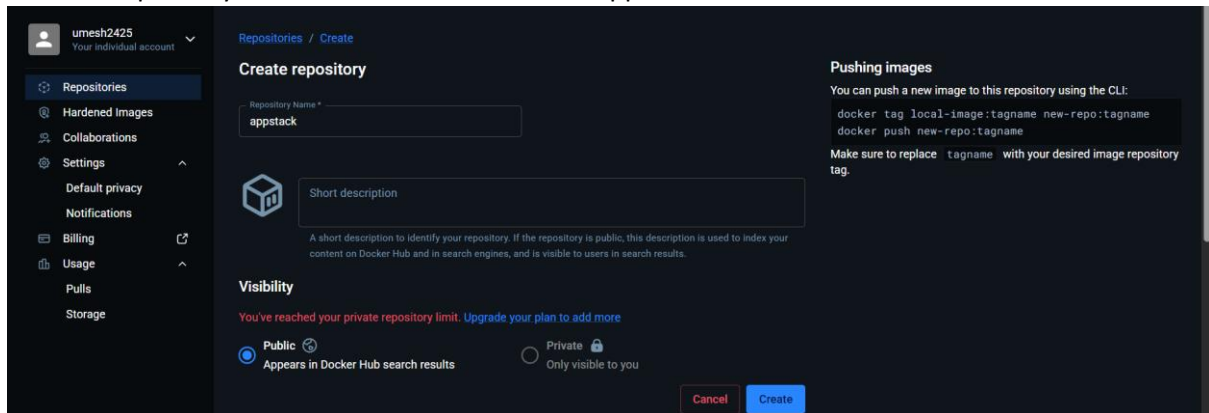
Username: umesh2425
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

Verify username →→→→ **docker info | grep username**

```
[root@jenkins Deployment-Pipeline]# docker info | grep Username
Username: umesh2425
```

Create a repository in Docker Hub with the name appstack.



Before building the Docker images

Add Jenkins user to docker group

usermod -aG docker Jenkins

id jenkins

```
[root@jenkins ~]# usermod -aG docker jenkins
[root@jenkins ~]# id jenkins
uid=993(jenkins) gid=993(jenkins) groups=993(jenkins),992(docker)
```

And restart Jenkins once

sudo systemctl restart Jenkins

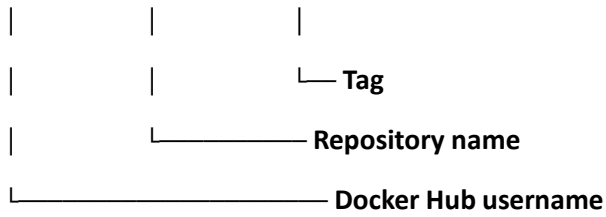
```
[root@jenkins ~]# sudo systemctl restart jenkins
[root@jenkins ~]#
```

Docker Section

Stage : Docker Build

```
stage('Docker Build') {
    steps {
        sh 'docker build -t umesh2425/appstack:app -f Docker-app/Dockerfile .'
        sh 'docker build -t umesh2425/appstack:db -f Docker-db/Dockerfile .'
    }
}
```

umesh2425 / appstack : app



Trigger the Pipeline

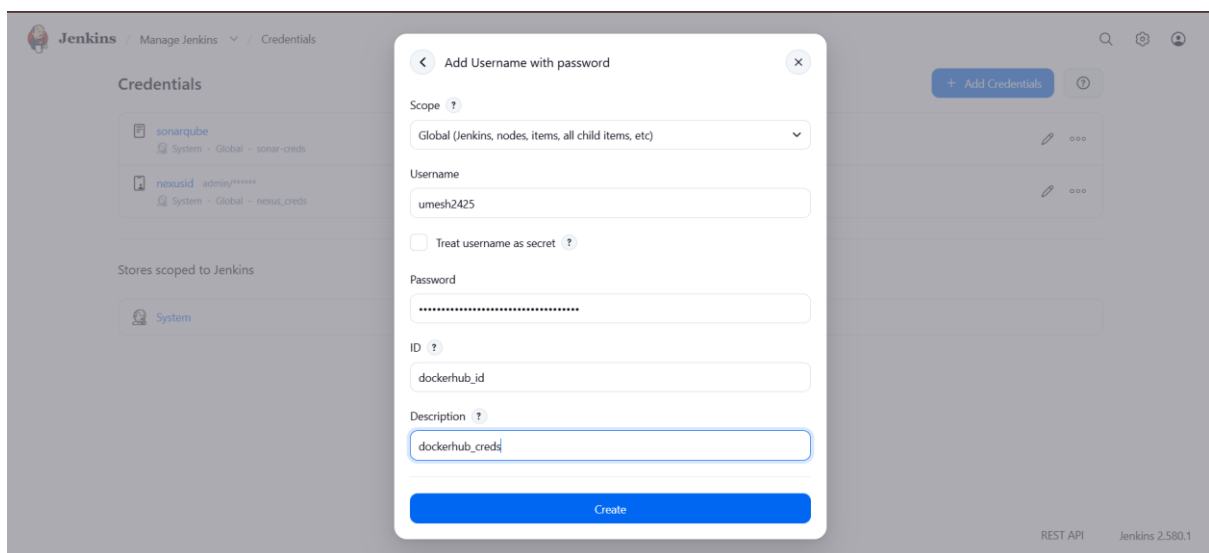
	Declarative: Tool Install	Code Checkout	Build and Test	Code Quality Analysis	Code Quality Gates	Artifact Uploader	Docker Build
stage times: time: ~51s)	90ms	1s	13s	30s	139ms	562ms	3s
	72ms	4s	13s	29s	151ms (paused for 456ms)	812ms	7s

Verify the Docker images in Jenkins after the pipeline completes.

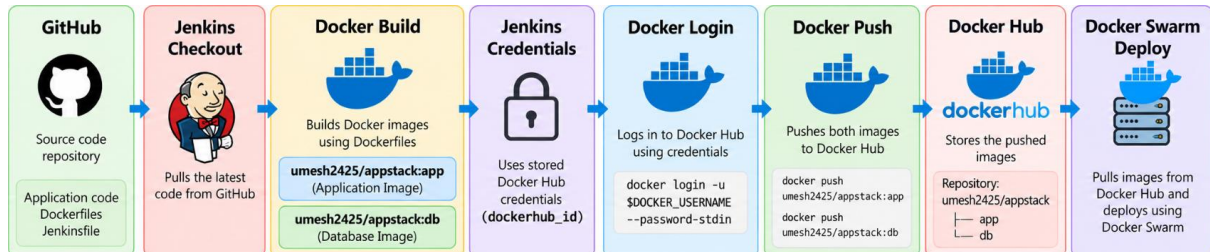
Stage: Docker push

Dockerhub login through Jenkins

Add Credentials in Manage Jenkins >> Credentials >>



After successfully building the images, Jenkins retrieves the Docker Hub credentials stored securely in Jenkins Credentials using `withCredentials()`, logs in to Docker Hub using the username and access token, and pushes both images to the appstack repository. The images are then available in Docker Hub and can be pulled by Docker Swarm during deployment.



For a clear and detailed explanation, refer to **Docker-Push-Stage-Explanation.md** file

Syntax for Pipeline

```
stage('Docker Push') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'dockerhub_id',
                usernameVariable: 'DOCKER_USERNAME',
                passwordVariable: 'DOCKER_PASSWORD'
            )
        ]) {
            sh '''
            echo "$DOCKER_PASSWORD" | docker login -u "$DOCKER_USERNAME" --password-stdin
            docker push umesh2425/appstack:app
            docker push umesh2425/appstack:db
            docker logout
            '''
        }
    }
}
```

Trigger the Pipeline

	Declarative: Tool Install	Code Checkout	Build and Test	Code Quality Analysis	Code Quality Gates	Artifact Uploader	Docker Build	Docker Push
Average stage times: (full run time: ~1min 2s)	73ms	1s	13s	29s	131ms	620ms	3s	6s
#29 Oct 04 01:21 No Changes	71ms	769ms	13s	30s	140ms (paused for 1s)	597ms	2s	34s

Verify Docker images in DockerHub

The screenshot shows the Docker Hub interface for the repository 'umesh2425/appstack'. The page includes a sidebar with navigation options like Repositories, Hardened Images, and Settings. The main content area shows the repository details, including the last push time (9 minutes ago) and repository size (262.1 MB). A table lists the tags, showing two tags: 'db' and 'app', both pushed 9 minutes ago. The right sidebar features a 'Docker commands' section with a 'Public view' button and a 'docker push' command. Below this, there is a 'Build with Docker Build Cloud' section with a 'Go to Docker Build Cloud' link.

Deploy Application

1) Initialize Docker Swarm in Jenkins Server

docker swarm init

2) Check Swarm node status:

docker node ls

```
[root@jenkins /]# docker swarm init
Swarm initialized: current node (y54f2ifmhr3m1twj2n18jbcq) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-5qf8f7zsqqlks1hk0b8595tfo5t8muguesbto5qm3iej7op9tu-f1afohtmcvskkazgm6jrpj12 172.31.46.111:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

[root@jenkins /]# docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
y54f2ifmhr3m1twj2n18jbcq *   jenkins        Ready     Active           Leader             25.0.16
```

Stage Deploy:

```
stage('Deploy') {  
    steps {  
        sh 'docker stack deploy -c compose.yml appstack'  
    }  
}
```

docker stack ls

```
[root@jenkins Deployment-Pipeline]# docker stack ls  
NAME                SERVICES  
appstack            2
```

docker stack services appstack

```
[root@jenkins ~]# docker stack services appstack  
ID                NAME                MODE                REPLICAS    IMAGE                PORTS  
90bwrq0yw4yq     appstack_application replicated          3/3          umesh2425/appstack:app *:1111->8080/tcp  
yyoxl9qewmev     appstack_devopsdb   replicated          1/1          umesh2425/appstack:db *:3306->3306/tcp
```

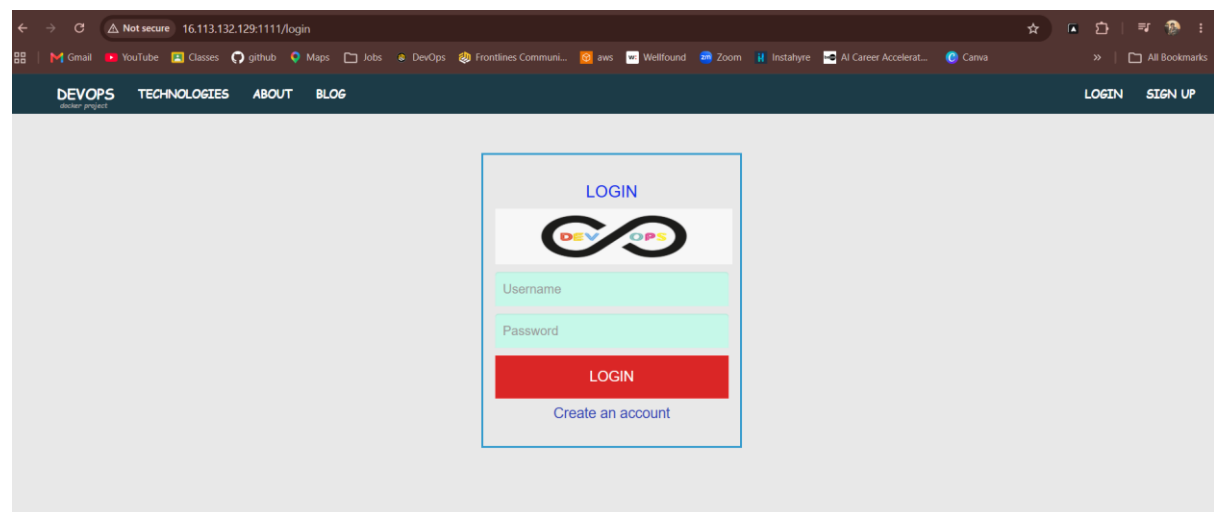
Trigger the Pipeline



Access the Application

Make sure to open the port 1111 in EC2 Jenkins Server (Inbound rules)

Open port which you have given in compose file



docker stack deploy -c compose.yml appstack

```
appstack          ← Stack
|
├── devopsdb      ← DB service
|   └── appstack:db
|
└── application   ← Application service
    └── appstack:app
```

appstack → stack name

devopsdb → database service

application → application service

umesh2425/appstack:db → DB Docker image

umesh2425/appstack:app → application Docker image

Think of it like:

compose.yml

↓

docker stack deploy

↓

appstack ← we give the stack its name here

├── devopsdb

└── application
